[5]

```
#EXERCISE 1 - DATA PREPROCESSING AND CLEANING
#MOHANASANKARI A
#241801168

from google.colab import drive
drive.mount('/content/drive')

import pandas as pd

df = pd.read_csv('/content/drive/MyDrive/Titanic-Dataset.csv')
```

```
Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.mount("/content/drive", force_remount=True).
```

[6]

```
#MOHANASANKARI A
#241801168
# Display the first few rows
print("First 5 Rows:")
print(df.head())
```

```
First 5 Rows:
   PassengerId  Survived  Pclass  \
0            1         0       3
1            2         1       1
2            3         1       3
3            4         1       1
4            5         0       3

                                                Name     Sex   Age  SibSp  \
0                            Braund, Mr. Owen Harris    male  22.0      1
1  Cumings, Mrs. John Bradley (Florence Briggs Th...  female  38.0      1
2                             Heikkinen, Miss. Laina  female  26.0      0
3       Futrelle, Mrs. Jacques Heath (Lily May Peel)  female  35.0      1
4                           Allen, Mr. William Henry    male  35.0      0

   Parch            Ticket     Fare Cabin Embarked
0      0         A/5 21171   7.2500   NaN        S
1      0          PC 17599  71.2833   C85        C
2      0  STON/O2. 3101282   7.9250   NaN        S
3      0            113803  53.1000  C123        S
4      0            373450   8.0500   NaN        S
```

[7]

```
#MOHANASANKARI A
#241801168
# Get dataset information
print("\nDataset Information:")
print(df.info())
```

```
Dataset Information:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 891 entries, 0 to 890
Data columns (total 12 columns):
 #   Column       Non-Null Count  Dtype
---  ------       --------------  -----
 0   PassengerId  891 non-null    int64
 1   Survived     891 non-null    int64
 2   Pclass       891 non-null    int64
 3   Name         891 non-null    object
 4   Sex          891 non-null    object
 5   Age          714 non-null    float64
 6   SibSp        891 non-null    int64
 7   Parch        891 non-null    int64
 8   Ticket       891 non-null    object
 9   Fare         891 non-null    float64
 10  Cabin        204 non-null    object
 11  Embarked     889 non-null    object
dtypes: float64(2), int64(5), object(5)
memory usage: 83.7+ KB
None
```

```
#MOHANASANKARI A
#241801168
# Display missing values
print("\nMissing Values:")
print(df.isnull().sum())
```

```
Missing Values:
PassengerId      0
Survived         0
Pclass           0
Name             0
Sex              0
Age            177
SibSp            0
Parch            0
Ticket           0
Fare             0
Cabin          687
Embarked         2
dtype: int64
```

```python
#MOHANASANKARI A
#241801168
# Forward fill and backward fill on 'Age'
df['Age'] = df['Age'].ffill().bfill()

print("Missing values in Age after filling:")
print(df['Age'].isnull().sum())
```

```
Missing values in Age after filling:
0
```

```python
#MOHANASANKARI A
#241801168
# Fill missing values in 'Cabin' with 'unknown' (limit = 5)
df['Cabin'] = df['Cabin'].fillna('unknown', limit=5)

print(df['Cabin'].head(10))
```

```
0    unknown
1        C85
2    unknown
3       C123
4    unknown
5    unknown
6        E46
7    unknown
8        NaN
9        NaN
Name: Cabin, dtype: object
```

```python
#MOHANASANKARI A
#241801168
# Remove duplicate records
duplicates = df.duplicated().sum()
print("Duplicate Records:", duplicates)

df = df.drop_duplicates()

print("Shape after removing duplicates:", df.shape)
```

```
Duplicate Records: 0
Shape after removing duplicates: (891, 12)
```

```python
#MOHANASANKARI A
#241801168
# Encode categorical column 'Sex' using LabelEncoder
from sklearn.preprocessing import LabelEncoder

le = LabelEncoder()

df['Sex'] = le.fit_transform(df['Sex'])

print(df[['Sex']].head())
```

```
   Sex
0    1
1    0
2    0
3    0
4    1
```

```python
#MOHANASANKARI A
#241801168
# Scale numerical feature 'Fare' using StandardScaler
from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()

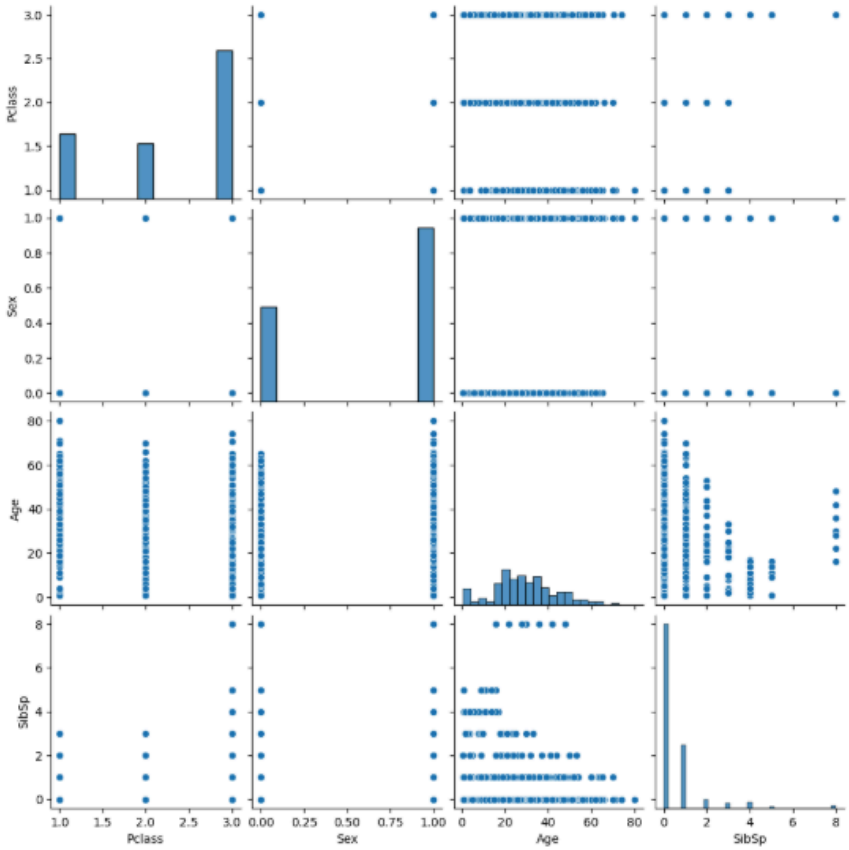
df[['Fare']] = scaler.fit_transform(df[['Fare']])

print(df[['Fare']].head())
```

```
       Fare
0 -0.502445
1  0.786845
2 -0.488854
3  0.420730
4 -0.486337
```

```
#MOHANASANKARI A
#241801168
# Pair Plot of selected features
import seaborn as sns
import matplotlib.pyplot as plt

sns.pairplot(df[['Pclass', 'Sex', 'Age', 'SibSp']])
plt.show()
```

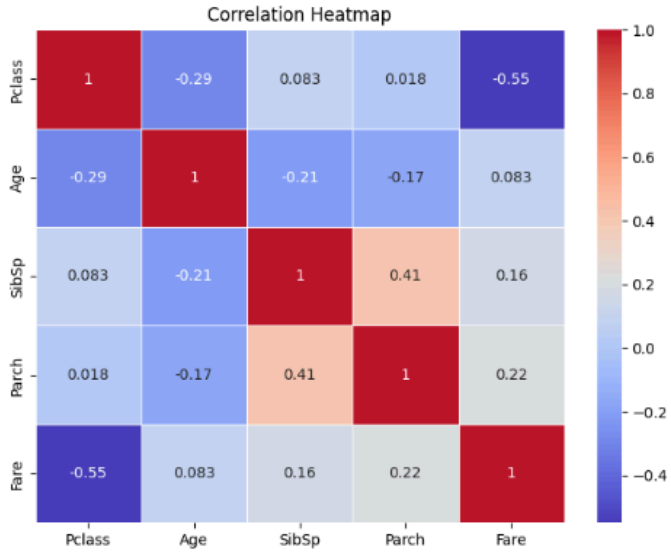


```python
#MOHANASANKARI A
#241801168
# Correlation Heatmap
plt.figure(figsize=(8,6))

corr = df[['Pclass', 'Age', 'SibSp', 'Parch', 'Fare']].corr()

sns.heatmap(corr,
            annot=True,
            cmap='coolwarm',
            linewidths=0.5)

plt.title("Correlation Heatmap")
plt.show()
```



```
# MOHANASANKARI A
# 241801168
#EX 2: DATA DISCOVERY AND EXPLORATORY DATA ANALYSIS
# TITANIC DATASET
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
from sklearn.impute import SimpleImputer
from sklearn.model_selection import train_test_split
```

```
# MOHANASANKARI A
# 241801168
# 1. Use titanic.csv and convert it into dataframe
df = pd.read_csv("titanic.csv")
print("Titanic Dataset:")
print(df.head())
```

```
Titanic Dataset:
   PassengerId  Survived  Pclass  \
0            1         0       3
1            2         1       1
2            3         1       3
3            4         1       1
4            5         0       3

                                                Name     Sex   Age  SibSp  \
0                            Braund, Mr. Owen Harris    male  22.0      1
1  Cumings, Mrs. John Bradley (Florence Briggs Th...  female  38.0      1
2                             Heikkinen, Miss. Laina  female  26.0      0
3       Futrelle, Mrs. Jacques Heath (Lily May Peel)  female  35.0      1
4                           Allen, Mr. William Henry    male  35.0      0

   Parch            Ticket     Fare Cabin Embarked
0      0         A/5 21171   7.2500   NaN        S
1      0          PC 17599  71.2833   C85        C
2      0  STON/O2. 3101282   7.9250   NaN        S
3      0            113803  53.1000  C123        S
4      0            373450   8.0500   NaN        S
```

```
# MOHANASANKARI A
# 241801168
# 2. Understand the data (shape, info, describe)
print("\nShape of Dataset:")
print(df.shape)
print("\nDataset Information:")
print(df.info())
print("\nStatistical Description:")
print(df.describe())
```

```
Shape of Dataset:
(891, 12)

Dataset Information:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 891 entries, 0 to 890
Data columns (total 12 columns):
 #   Column       Non-Null Count  Dtype
---  ------       --------------  -----
 0   PassengerId  891 non-null    int64
 1   Survived     891 non-null    int64
 2   Pclass       891 non-null    int64
```

```
# MOHANASANKARI A
# 241801168
# 3. Fill missing values in Age using SimpleImputer
imputer = SimpleImputer(strategy="mean")
df["Age"] = imputer.fit_transform(df[["Age"]]).ravel()
print("\nMissing Age values after imputation:")
print(df["Age"].isnull().sum())
```

```
Missing Age values after imputation:
0
```

```
# MOHANASANKARI A
# 241801168
# 4. Fill missing Cabin with "Unknown"
df["Cabin"] = df["Cabin"].fillna("Unknown")
print("\nMissing Cabin values:")
print(df["Cabin"].isnull().sum())
```

```
Missing Cabin values:
0
```

```
# MOHANASANKARI A
# 241801168
# 5. Fill missing Embarked with the mode
df["Embarked"] = df["Embarked"].fillna(df["Embarked"].mode()[0])
print("\nMissing Embarked values:")
print(df["Embarked"].isnull().sum())
```

```
Missing Embarked values:
0
```

```python
# MOHANASANKARI A
# 241801168
# 6. Analyse Passenger Class using countplot
plt.figure(figsize=(7, 5))
sns.countplot(x="Pclass", data=df)
plt.title("Passenger Class Distribution")
plt.xlabel("Passenger Class")
plt.ylabel("Number of Passengers")
plt.show()
```




```python
# MOHANASANKARI A
# 241801168
# 7. List names of passengers who are female and survived
female_survived = df[
    (df["Sex"] == "female") &
    (df["Survived"] == 1)
]
print("\n7. Female Passengers Who Survived:")
print(female_survived["Name"].to_string(index=False))
```

```
7. Female Passengers Who Survived:
Cumings, Mrs. John Bradley (Florence Briggs Tha...
                            Heikkinen, Miss. Laina
      Futrelle, Mrs. Jacques Heath (Lily May Peel)
 Johnson, Mrs. Oscar W (Elisabeth Vilhelmina Berg)
               Nasser, Mrs. Nicholas (Adele Achem)
                   Sandstrom, Miss. Marguerite Rut
                          Bonnell, Miss. Elizabeth
                  Hewlett, Mrs. (Mary D Kingcome)
```

```python
# MOHANASANKARI A
# 241801168
# 8. Find all passengers who are in 3rd class
#    and under the age of 18
third_class_under18 = df[
    (df["Pclass"] == 3) &
    (df["Age"] < 18)
]
print("\n8. 3rd Class Passengers Under Age 18:")
print(third_class_under18[["Name", "Age", "Pclass"]].to_string(index=False))
```

```
                            Calic, Mr. Jovo 17.00       3
                    Panula, Master. Eino Viljami  1.00       3
Goldsmith, Master. Frank John William "Frankie"  9.00       3
                           Rice, Master. Arthur  4.00       3
                    Johnson, Miss. Eleanor Ileen  1.00       3
          Asplund, Master. Clarence Gustaf Hugo  9.00       3
            Kink-Heilmann, Miss. Luise Gretchen  4.00       3
                    Strom, Miss. Telma Matilda  2.00       3
                      Carr, Miss. Helen "Ellen" 16.00       3
                Sunderland, Mr. Victor Francis 16.00       3
```

```python
# MOHANASANKARI A
# 241801168
# 9. Find passengers who are in class 1
#    and older than 40
class1_over40 = df[
    (df["Pclass"] == 1) &
    (df["Age"] > 40)
]
print("\n9. Class 1 Passengers Older Than 40:")
print(class1_over40[["Name", "Age", "Pclass"]].to_string(index=False))
```

```
9. Class 1 Passengers Older Than 40:
                                                Name  Age  Pclass
                                    McCarthy, Mr. Timothy J 54.0       1
                                   Bonnell, Miss. Elizabeth 58.0       1
                             Holverson, Mr. Alexander Oskar 42.0       1
                 Harper, Mrs. Henry Sleeper (Myna Haxtun) 49.0       1
                             Ostby, Mr. Engelhart Cornelius 65.0       1
                                Harris, Mr. Henry Birkhardt 45.0       1
```

```python
# MOHANASANKARI A
# 241801168
class1_over40_survived = df[
    (df["Pclass"] == 1) &
    (df["Age"] > 40) &
    (df["Survived"] == 1)
]

print("\n10. Class 1 Passengers Older Than 40 Who Survived:")
print(class1_over40_survived[
    ["Name", "Age", "Pclass", "Survived"]
].to_string(index=False))
```

```
10. Class 1 Passengers Older Than 40 Who Survived:
                                                Name  Age  Pclass  Survived
                                  Bonnell, Miss. Elizabeth 58.0       1         1
                       Harper, Mrs. Henry Sleeper (Myna Haxtun) 49.0       1         1
                  Romaine, Mr. Charles Hallace ("Mr C Rolmane") 45.0       1         1
                      Brown, Mrs. James Joseph (Margaret Tobin) 44.0       1         1
                                   Lurette. Miss. Elise 58.0       1         1
```

```python
# MOHANASANKARI A
# 241801168
male_fare100 = df[
    (df["Sex"] == "male") &
    (df["Fare"] > 100)
]

print("\n11. Male Passengers Who Paid Fare > 100:")
print(male_fare100[
    ["Name", "Fare", "Sex"]
].to_string(index=False))
```

output

```
11. Male Passengers Who Paid Fare > 100:
                                Name     Fare  Sex
          Fortune, Mr. Charles Alexander 263.0000 male
                Baxter, Mr. Quigg Edmond 247.5208 male
          Allison, Master. Hudson Trevor 151.5500 male
```

```python
# MOHANASANKARI A
# 241801168
portC_class2 = df[
    (df["Embarked"] == "C") &
    (df["Pclass"] == 2)
]

print("\n12. Passengers Embarked from C and in Class 2:")
print(portC_class2[
    ["Name", "Embarked", "Pclass"]
].to_string(index=False))
```

```
12. Passengers Embarked from C and in Class 2:
                                        Name Embarked  Pclass
             Nasser, Mrs. Nicholas (Adele Achem)        C       2
Laroche, Miss. Simonne Marie Anne Andree        C       2
                      Nasser, Mr. Nicholas        C       2
                        Richard, Mr. Emile        C       2
```

```python
# MOHANASANKARI A
# 241801168
sibsp_more2 = df[df["SibSp"] > 2]

print("\n13. Passengers with SibSp > 2:")
print(sibsp_more2[
    ["Name", "SibSp"]
].to_string(index=False))
```

```
13. Passengers with SibSp > 2:
                                Name  SibSp
          Palsson, Master. Gosta Leonard      3
                    Rice, Master. Eugene      4
           Palsson, Miss. Torborg Danira      3
          Fortune, Mr. Charles Alexander      3
              Panula, Master. Juha Niilo      4
```

```python
# MOHANASANKARI A
# 241801168
oldest_survived = df[
    df["Survived"] == 1
].sort_values(
    by="Age",
    ascending=False
).head(5)

print("\n15. Top 5 Oldest Passengers Who Survived:")
print(oldest_survived[
    ["Name", "Age", "Survived"]
].to_string(index=False))
```

```
15. Top 5 Oldest Passengers Who Survived:
                                    Name  Age  Survived
      Barkworth, Mr. Algernon Henry Wilson 80.0         1
         Andrews, Miss. Kornelia Theodosia 63.0         1
                     Turkula, Mrs. (Hedwig) 63.0         1
                           Harris, Mr. George 62.0         1
Stone, Mrs. George Nelson (Martha Evelyn) 62.0         1
```

```python
# MOHANASANKARI A
# 241801168
zero_fare = df[df["Fare"] == 0]

print("\n16. Passengers Who Paid Zero Fare:")
print(zero_fare[
    ["Name", "Fare"]
].to_string(index=False))
```

```
16. Passengers Who Paid Zero Fare:
                          Name  Fare
              Leonard, Mr. Lionel   0.0
            Harrison, Mr. William   0.0
    Tornquist, Mr. William Henry   0.0
     Parkes, Mr. Francis "Frank"   0.0
 Johnson, Mr. William Cahoone Jr   0.0
  Cunningham, Mr. Alfred Fleming   0.0
           Campbell, Mr. William   0.0
Frost, Mr. Anthony Wood "Archie"   0.0
             Johnson, Mr. Alfred   0.0
```

```python
# MOHANASANKARI A
# 241801168
# Features and target
X = df.drop("Survived", axis=1)
y = df["Survived"]
# Select numerical columns for a simple train-test split
X_numeric = df[
    ["Pclass", "Age", "SibSp", "Parch", "Fare"]
]
# Split into 80% training and 20% testing
X_train, X_test, y_train, y_test = train_test_split(
    X_numeric,
    y,
    test_size=0.2,
    random_state=42
)
print("\n17. Train-Test Split:")
print("Training data:", X_train.shape)
print("Testing data :", X_test.shape)
print("Training target:", y_train.shape)
print("Testing target :", y_test.shape)
```

```
17. Train-Test Split:
Training data: (712, 5)
Testing data : (179, 5)
Training target: (712,)
Testing target : (179,)
```

```python
# MOHANASANKARI A
# 241801168
print("\nFinal Dataset:")
print(df.head())

print("\nMissing Values After Preprocessing:")
print(df.isnull().sum())
```

```
...
Final Dataset:
   PassengerId  Survived  Pclass  \
0            1         0       3
1            2         1       1
2            3         1       3
3            4         1       1
4            5         0       3
```

```python
# MOHANASANKARI A
# 241801168
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.preprocessing import PolynomialFeatures
from sklearn.pipeline import Pipeline
from sklearn.svm import SVR
from sklearn.cluster import KMeans
from sklearn.metrics import mean_absolute_error, mean_squared_error
```

```python
# MOHANASANKARI A
#241801168
# Load the Advertising dataset
df = pd.read_csv("advertising.csv")
# Display first 5 rows
df.head()
```

| | TV | Radio | Newspaper | Sales |
|---|---|---|---|---|
| 0 | 230.1 | 37.8 | 69.2 | 22.1 |
| 1 | 44.5 | 39.3 | 45.1 | 10.4 |
| 2 | 17.2 | 45.9 | 69.3 | 12.0 |
| 3 | 151.5 | 41.3 | 58.5 | 16.5 |
| 4 | 180.8 | 10.8 | 58.4 | 17.9 |

Next steps: Generate code with df | New interactive sheet

```python
# MOHANASANKARI A
#241801168
# Display dataset information
print("Dataset Information:")
df.info()
print("\nShape of Dataset:", df.shape)
print("\nColumn Names:")
print(df.columns)
print("\nFirst 5 Rows:")
display(df.head())
```

```
Dataset Information:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 200 entries, 0 to 199
```

```python
# MOHANASANKARI A
# 241801168
# Check for missing/null values
print("Null values in each column:")
print(df.isnull().sum())
```

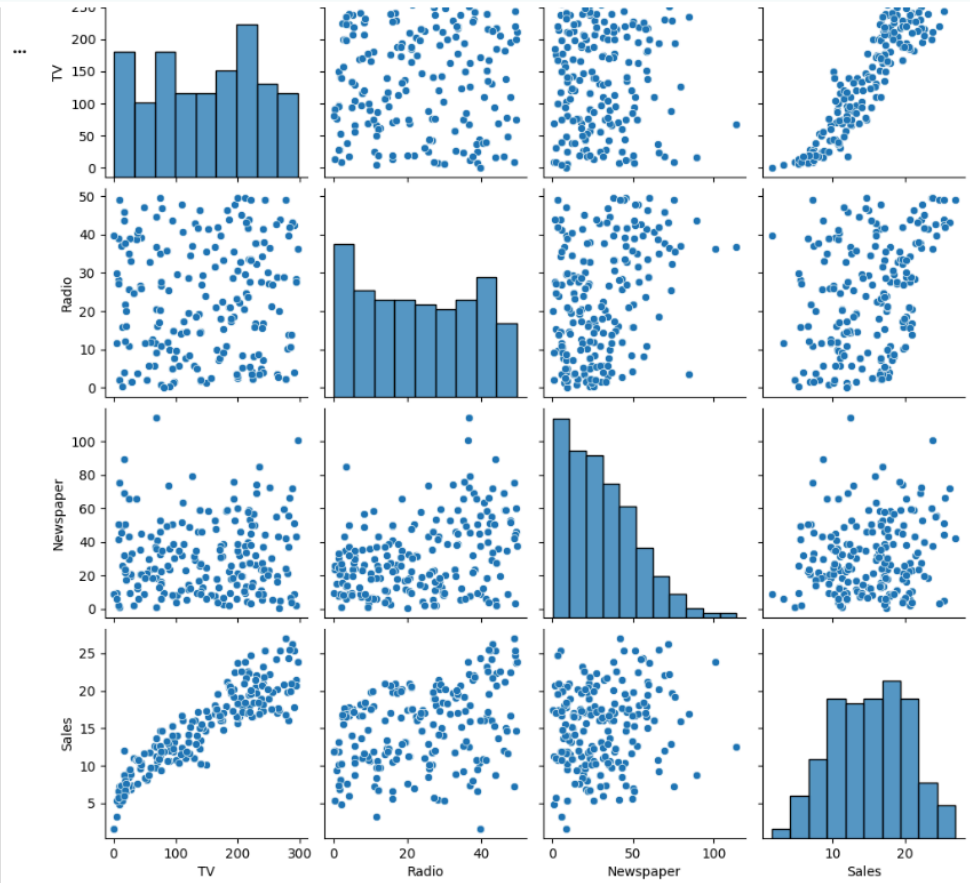
```
Null values in each column:
TV          0
Radio       0
Newspaper   0
Sales       0
dtype: int64
```

```python
# MOHANASANKARI A
#241801168
# Display data types
print("Data types of each column:")
print(df.dtypes)
# Select features
X = df[["TV", "Radio", "Newspaper"]]
# Select target
y = df["Sales"]
print("Features:")
display(X.head())
print("\nTarget:")
display(y.head())
sns.pairplot(df[["TV", "Radio", "Newspaper", "Sales"]])
plt.show()
```

```
Data types of each column:
TV          float64
Radio       float64
Newspaper   float64
Sales       float64
dtype: object
Features:
```

| | TV | Radio | Newspaper |
|---|---|---|---|
| 0 | 230.1 | 37.8 | 69.2 |
| 1 | 44.5 | 39.3 | 45.1 |
| 2 | 17.2 | 45.9 | 69.3 |
| 3 | 151.5 | 41.3 | 58.5 |
| 4 | 180.8 | 10.8 | 58.4 |

```
Target:
```




```
# MOHANASANKARI A
#241801168
# Split the dataset into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
print("Training feature shape:", X_train.shape)
print("Testing feature shape:", X_test.shape)
print("Training target shape:", y_train.shape)
print("Testing target shape:", y_test.shape)
```

```
Training feature shape: (160, 3)
Testing feature shape: (40, 3)
Training target shape: (160,)
Testing target shape: (40,)
```

```
# MOHANASANKARI A
#241801168
# Create Linear Regression model
linear_model = LinearRegression()
# Fit the model using training data
linear_model.fit(X_train, y_train)
print("Linear Regression model trained successfully.")
y_pred_linear = linear_model.predict(X_test)
print("Linear Regression Predictions:")
print(y_pred_linear[:10])
```

```
Linear Regression model trained successfully.
Linear Regression Predictions:
[17.0347724  20.40974033 23.72398873  9.27278518 21.68271879 12.56940161
 21.08119452  8.69035045 17.23701254 16.66657475]
```

```
# MOHANASANKARI A
#241801168
# Create Polynomial Regression pipeline with degree 2
poly_model = Pipeline([
    ("polynomial_features", PolynomialFeatures(degree=2)),
    ("linear_regression", LinearRegression())
])
# Fit the polynomial regression model
poly_model.fit(X_train, y_train)
print("Polynomial Regression model trained successfully.")
# Predict Sales using Polynomial Regression
y_pred_poly = poly_model.predict(X_test)
print("Polynomial Regression Predictions:")
print(y_pred_poly[:10])
```

```
Polynomial Regression model trained successfully.
Polynomial Regression Predictions:
[17.562438   21.96285664 22.07580481  7.34916293 23.23684435 12.37109504
 22.12872506  8.76904874 17.01492635 16.914806081
```

```python
# MOHANASANKARI A
#241801168
# Create SVR model with RBF kernel
svr_model = SVR(kernel="rbf")
# Fit SVR model
svr_model.fit(X_train, y_train)
print("SVR model trained successfully.")
# Predict Sales using SVR
y_pred_svr = svr_model.predict(X_test)
print("SVR Predictions:")
print(y_pred_svr[:10])
```

```
SVR model trained successfully.
SVR Predictions:
[17.15330196 19.49875585 20.96245814  8.60209215 19.88553201 12.03803929
 19.98237498  9.22752786 17.73379117 17.08961377]
```

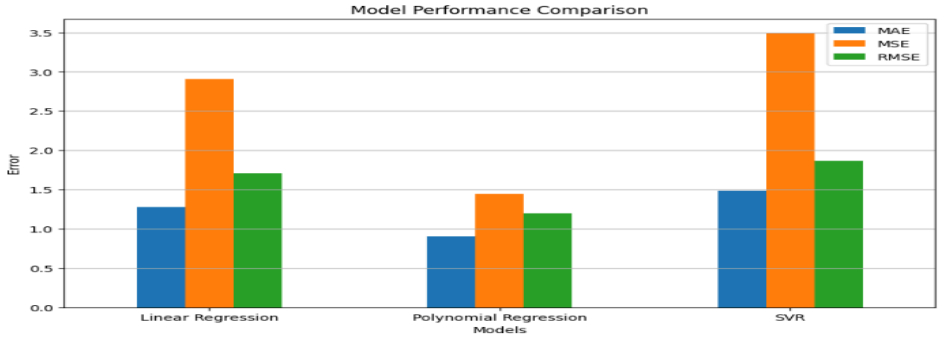
```python
# MOHANASANKARI A, 241801168
# Linear Regression metrics
mae_linear = mean_absolute_error(y_test, y_pred_linear)
mse_linear = mean_squared_error(y_test, y_pred_linear)
rmse_linear = np.sqrt(mse_linear)
# Polynomial Regression metrics
mae_poly = mean_absolute_error(y_test, y_pred_poly)
mse_poly = mean_squared_error(y_test, y_pred_poly)
rmse_poly = np.sqrt(mse_poly)
# SVR metrics
mae_svr = mean_absolute_error(y_test, y_pred_svr)
mse_svr = mean_squared_error(y_test, y_pred_svr)
rmse_svr = np.sqrt(mse_svr)
print("Linear Regression")
print("MAE :", mae_linear)
print("MSE :", mse_linear)
print("RMSE:", rmse_linear)
print("\nPolynomial Regression")
print("MAE :", mae_poly)
print("MSE :", mse_poly)
print("RMSE:", rmse_poly)
print("\nSVR")
print("MAE :", mae_svr)
print("MSE :", mse_svr)
print("RMSE:", rmse_svr)
```

```
Linear Regression
MAE : 1.2748262109549338
MSE : 2.9077569102710896
RMSE: 1.7052146229349223

Polynomial Regression
MAE : 0.9033713382792442
MSE : 1.44254263690159
RMSE: 1.2010589647896517

SVR
MAE : 1.4836223881587627
MSE : 3.499087197255254
RMSE: 1.8705847206836834
```

| | Model | MAE | MSE | RMSE |
|---|---|---|---|---|
| 0 | Linear Regression | 1.274826 | 2.907757 | 1.705215 |
| 1 | Polynomial Regression | 0.903371 | 1.442543 | 1.201059 |
| 2 | SVR | 1.483622 | 3.499087 | 1.870585 |




Next steps: Generate code with results | New interactive sheet

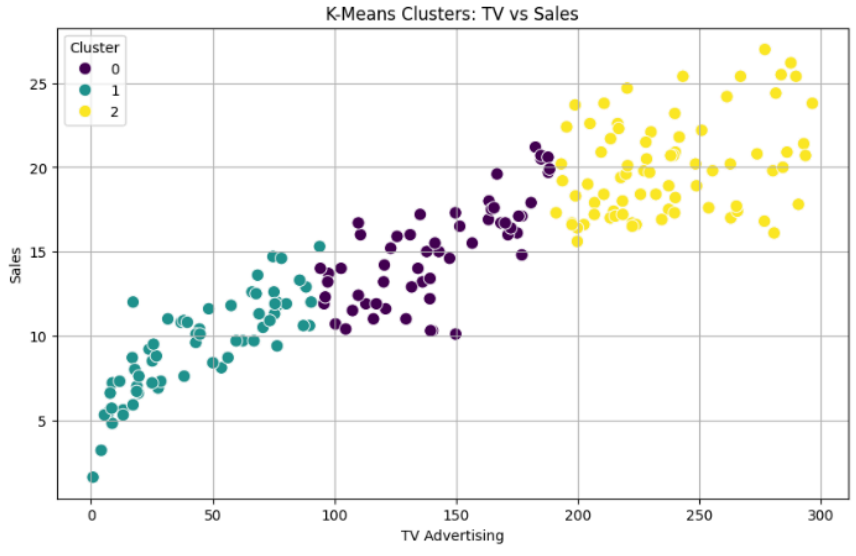
```python
# MOHANASANKARI A
#241801168
# Create K-Means model with 3 clusters
kmeans = KMeans(n_clusters=3, random_state=42, n_init=10)
# Fit K-Means on feature set
kmeans.fit(X)
print("K-Means clustering completed successfully.")
df["Cluster"] = kmeans.labels_
# Display dataset with cluster labels
display(df.head(10))
print("Number of data points in each cluster:")
print(df["Cluster"].value_counts().sort_index())
```

```
K-Means clustering completed successfully.
```

| | TV | Radio | Newspaper | Sales | Cluster |
|---|---|---|---|---|---|
| 0 | 230.1 | 37.8 | 69.2 | 22.1 | 2 |
| 1 | 44.5 | 39.3 | 45.1 | 10.4 | 1 |
| 2 | 17.2 | 45.9 | 69.3 | 12.0 | 1 |
| 3 | 151.5 | 41.3 | 58.5 | 16.5 | 0 |
| 4 | 180.8 | 10.8 | 58.4 | 17.9 | 0 |
| 5 | 8.7 | 48.9 | 75.0 | 7.2 | 1 |
| 6 | 57.5 | 32.8 | 23.5 | 11.8 | 1 |
| 7 | 120.2 | 19.6 | 11.6 | 13.2 | 0 |
| 8 | 8.6 | 2.1 | 1.0 | 4.8 | 1 |
| 9 | 199.8 | 2.6 | 21.2 | 15.6 | 2 |

```
Number of data points in each cluster:
Cluster
0    58
1    65
2    77
Name: count, dtype: int64
```

```python
# MOHANASANKARI A
#241801168
# Visualize clusters using TV vs Sales
plt.figure(figsize=(10, 6))
sns.scatterplot(
    data=df,
    x="TV",
    y="Sales",
    hue="Cluster",
    palette="viridis",
    s=80
)
plt.title("K-Means Clusters: TV vs Sales")
plt.xlabel("TV Advertising")
plt.ylabel("Sales")
plt.legend(title="Cluster")
plt.grid(True)
plt.show()
```



```python
# MOHANASANKARI A
#241801168
# Display the cluster centers
cluster_centers = pd.DataFrame(
    kmeans.cluster_centers_,
    columns=["TV", "Radio", "Newspaper"]
)
print("Cluster Centers:")
display(cluster_centers)
```

```
Cluster Centers:
```

| | TV | Radio | Newspaper |
|---|---|---|---|
| 0 | 141.796552 | 20.779310 | 26.181034 |
| 1 | 45.209231 | 23.427692 | 31.064615 |
| 2 | 236.957143 | 24.997403 | 33.416883 |

Next steps: Generate code with cluster_centers | New interactive sheet

```python
# MOHANASANKARI A
#241801168
print("========== MODEL SUMMARY ==========")
print("\nLinear Regression:")
print("MAE  =", mae_linear)
print("MSE  =", mse_linear)
print("RMSE =", rmse_linear)
print("\nPolynomial Regression:")
print("MAE  =", mae_poly)
print("MSE  =", mse_poly)
print("RMSE =", rmse_poly)
print("\nSupport Vector Regression:")
print("MAE  =", mae_svr)
print("MSE  =", mse_svr)
print("RMSE =", rmse_svr)
print("\n========== CLUSTER SUMMARY ==========")
print(df["Cluster"].value_counts().sort_index())
print("\nExercise completed successfully!")
```

```
========== MODEL SUMMARY ==========

Linear Regression:
MAE  = 1.2748262109549338
MSE  = 2.9077569102710896
RMSE = 1.7052146229349223

Polynomial Regression:
MAE  = 0.9033713382792442
MSE  = 1.44254263690159

Support Vector Regression:
MAE  = 1.4836223881587627
MSE  = 3.499087197255254
RMSE = 1.8705847206836834

========== CLUSTER SUMMARY ==========
Cluster
0    58
1    65
2    77
Name: count, dtype: int64

Exercise completed successfully!
```

```python
#MOHANASANKARI A
#241801168
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics.pairwise import cosine_similarity
import numpy as np
```

```python
#MOHANASANKARI A
#241801168
documents = [
    "Information retrieval is the process of obtaining relevant information.",
    "Machine learning and deep learning are parts of artificial intelligence.",
    "Retrieval models use various ranking techniques.",
    "TF-IDF is a common method for text matching.",
    "Python is a popular language for machine learning."
]

print("Documents:")
for i, doc in enumerate(documents):
    print(f"Doc {i+1}: {doc}")
```

```
Documents:
Doc 1: Information retrieval is the process of obtaining relevant information.
Doc 2: Machine learning and deep learning are parts of artificial intelligence.
Doc 3: Retrieval models use various ranking techniques.
Doc 4: TF-IDF is a common method for text matching.
Doc 5: Python is a popular language for machine learning.
```

```python
#MOHANASANKARI A
#241801168
vectorizer = TfidfVectorizer()
tfidf_matrix = vectorizer.fit_transform(documents)
print("\nTF-IDF Matrix:")
print(tfidf_matrix)
```

```python
#MOHANASANKARI A
#241801168
print("\nFeature Names (Vocabulary):")

print(vectorizer.get_feature_names_out())
```

```
Feature Names (Vocabulary):
['and' 'are' 'artificial' 'common' 'deep' 'for' 'idf' 'information'
 'intelligence' 'is' 'language' 'learning' 'machine' 'matching' 'method'
 'models' 'obtaining' 'of' 'parts' 'popular' 'process' 'python' 'ranking'
 'relevant' 'retrieval' 'techniques' 'text' 'tf' 'the' 'use' 'various']
```

```python
#MOHANASANKARI A
#241801168
query = ["What is information retrieval in machine learning?"]
query_vec = vectorizer.transform(query)
print("\nQuery TF-IDF Vector:")
print(query_vec)
```

```
Query TF-IDF Vector:
<Compressed Sparse Row sparse matrix of dtype 'float64'
        with 5 stored elements and shape (1, 31)>
  Coords        Values
  (0, 7)        0.5422255279709232
  (0, 9)        0.36313475547801904
  (0, 11)       0.4374641418373903
  (0, 12)       0.4374641418373903
  (0, 24)       0.4374641418373903
```

```python
#MOHANASANKARI A
#241801168
cos_sim = cosine_similarity(query_vec, tfidf_matrix)
print("\nCosine Similarity:")
print(cos_sim)
```

```
Cosine Similarity:
[[0.53820999 0.33642487 0.14847233 0.09127334 0.40837224]]
```

```
#MOHANASANKARI A
#241801168
top_n = 3
top_indices = np.argsort(cos_sim[0])[::-1][:top_n]
print("\nTop Relevant Documents:")
for i in top_indices:
    print(f"Doc {i+1}: {documents[i]}")
```

```
Top Relevant Documents:
Doc 1: Information retrieval is the process of obtaining relevant information.
Doc 5: Python is a popular language for machine learning.
Doc 2: Machine learning and deep learning are parts of artificial intelligence.
```

```
#MOHANASANKARI A
#241801168
query = ["What is python"]
query_vec = vectorizer.transform(query)
cos_sim = cosine_similarity(query_vec, tfidf_matrix)
print("\nCosine Similarity for 'What is python':")
print(cos_sim)
```

```
Cosine Similarity for 'What is python':
[[0.11934505 0.         0.         0.13986295 0.51786157]]
```

```
#MOHANASANKARI A
#241801168
top_index = np.argmax(cos_sim[0])
print("\nTop Relevant Document:")
print(f"Doc {top_index+1}: {documents[top_index]}")
```

```
Top Relevant Document:
Doc 5: Python is a popular language for machine learning.
```

# Ex5a: INFORMATION EXTRACTION USING NLTK

### 1. Download Amazon Fine Food Reviews Dataset from Kaggle into a DataFrame.

```python
#Mohanasankari A
#241801168

!pip install nltk pandas kaggle -q

import pandas as pd
import nltk
import re
import string

from nltk.tokenize import word_tokenize
from nltk.corpus import stopwords
from nltk import pos_tag, ne_chunk

from google.colab import files
uploaded = files.upload()

df = pd.read_csv('Reviews.csv')

print(df.head())
```

### 2. Install and Import Required Libraries.

```python
#Mohanasankari A
#241801168
```

```
!pip install nltk pandas -q

import pandas as pd
import nltk
import re
import string

from nltk.tokenize import word_tokenize
from nltk.corpus import stopwords
from nltk import pos_tag, ne_chunk

print("Required libraries imported successfully")
```

### 3. Import NLTK Resources.

```
Python
#Mohanasankari A
#241801168

nltk.download('punkt')
nltk.download('punkt_tab')
nltk.download('stopwords')
nltk.download('averaged_perceptron_tagger')
nltk.download('averaged_perceptron_tagger_eng')
nltk.download('maxent_ne_chunker')
nltk.download('maxent_ne_chunker_tab')
nltk.download('words')

print("NLTK resources downloaded successfully")
```

### 4. Load the Dataset.

**a. Select the column containing review text.**

**b. Remove any missing or null entries.**

**c. Limit the dataset to 10,000 reviews for faster processing.**

```python
#Mohanasankari A
#241801168

df = pd.read_csv('Reviews.csv')

df = df[['Text']]

df = df.dropna(subset=['Text'])

df = df.head(10000)

print("Dataset loaded successfully")
print("Number of reviews:", len(df))
print(df.head())
```

## 5. Perform the Preprocessing Step.

### a. Lowercase.

### b. Remove punctuation.

```python
#Mohanasankari A
#241801168

def preprocess_text(text):
    text = text.lower()
    text = text.translate(
        str.maketrans('', '', string.punctuation)
    )
    return text

df['Cleaned_Text'] = df['Text'].apply(preprocess_text)

print(df[['Text', 'Cleaned_Text']].head())
```

## 6. Perform Tokenization and Remove Stopwords, Non-Alphabetic Tokens.

Python

```python
#Mohanasankari A
#241801168

stop_words = set(stopwords.words('english'))

def tokenize_text(text):
    tokens = word_tokenize(text)

    filtered_tokens = [
        word for word in tokens
        if word.isalpha() and word not in stop_words
    ]

    return filtered_tokens

df['Tokens'] = df['Cleaned_Text'].apply(tokenize_text)

print("Tokenized reviews:")
print(df['Tokens'].head())
```

### 7. Perform Parts of Speech (POS) Tagging.

Python

```python
#Mohanasankari A
#241801168

df['POS_Tagging'] = df['Tokens'].apply(pos_tag)

print("Parts of Speech Tagging:")
print(df['POS_Tagging'].head())
```

### 8. Identify Named Entity Recognition (NER) for the Separated Tokens.

Python

```python
#Mohanasankari A
#241801168
```

```
df['NER'] = df['POS_Tagging'].apply(ne_chunk)

def extract_entities(tree):
    entities = []

    for subtree in tree:
        if hasattr(subtree, 'label'):
            entity_name = ' '.join(
                word for word, tag in subtree.leaves()
            )
            entities.append((entity_name, subtree.label()))

    return entities

df['Named_Entities'] = df['NER'].apply(extract_entities)

print("Named Entity Recognition:")
print(df[['Tokens', 'POS_Tagging', 'Named_Entities']].head())
```

## RESULT

Thus, information extraction was performed successfully on the Amazon Fine Food Reviews dataset using NLTK. The reviews were preprocessed, tokenized, and filtered to remove stopwords and non-alphabetic tokens. Parts of Speech tagging and Named Entity Recognition were also performed successfully.

# Ex5b: INFORMATION EXTRACTION USING spaCy

### 1. Download Amazon Fine Food Reviews Dataset from Kaggle into a DataFrame.

Python

```
#Mohanasankari A
#241801168

!pip install pandas spacy -q

import pandas as pd
import spacy
import string
import re

from google.colab import files
uploaded = files.upload()

df = pd.read_csv('Reviews.csv')

print(df.head())
```

### 2. Install and Import Required Libraries.

Python

```
#Mohanasankari A
#241801168

!pip install pandas spacy -q

import pandas as pd
import spacy
import string
```

```
import re

print("Required libraries imported successfully")
```

### 3. Import NLTK Resources.

```
Python
#Mohanasankari A
#241801168

import nltk

nltk.download('stopwords')
nltk.download('punkt')
nltk.download('punkt_tab')

print("NLTK resources downloaded successfully")
```

### 4. Load the Dataset.

**a. Select the column containing review text.**

**b. Remove any missing or null entries.**

**c. Limit the dataset to 10,000 reviews for faster processing.**

```
Python
#Mohanasankari A
#241801168

df = pd.read_csv('Reviews.csv')

df = df[['Text']]

df = df.dropna(subset=['Text'])

df = df.head(10000)
```

```
print("Dataset loaded successfully")
print("Number of reviews:", len(df))
print(df.head())
```

## 5. Do the Preprocessing Step.

**a. Lowercase.**

**b. Remove punctuation.**

Python

```
#Mohanasankari A
#241801168

def preprocess_text(text):
    text = text.lower()
    text = text.translate(
        str.maketrans('', '', string.punctuation)
    )
    return text

df['Cleaned_Text'] = df['Text'].apply(preprocess_text)

print(df[['Text', 'Cleaned_Text']].head())
```

## 6. Perform Tokenization and Remove Stopwords, Non-Alphabetic Tokens.

Python

```
#Mohanasankari A
#241801168

nltk.download('stopwords')

from nltk.corpus import stopwords
```

```
stop_words = set(stopwords.words('english'))

def tokenize_text(text):
    tokens = text.split()

    filtered_tokens = [
        word for word in tokens
        if word.isalpha() and word not in stop_words
    ]

    return filtered_tokens

df['Tokens'] = df['Cleaned_Text'].apply(tokenize_text)

print("Tokenized reviews:")
print(df['Tokens'].head())
```

### 7. Perform Parts of Speech (POS) Tagging.

Python

```
#Mohanasankari A
#241801168

nlp = spacy.load("en_core_web_sm")

def pos_tagging(tokens):
    text = " ".join(tokens)
    doc = nlp(text)

    return [(token.text, token.pos_) for token in doc]

df['POS_Tagging'] = df['Tokens'].apply(pos_tagging)

print("Parts of Speech Tagging:")
print(df['POS_Tagging'].head())
```

## 8. Identify Named Entity Recognition (NER) for the Separated Tokens.

```python
#Mohanasankari A
#241801168

def extract_ner(tokens):
    text = " ".join(tokens)
    doc = nlp(text)

    entities = [
        (ent.text, ent.label_)
        for ent in doc.ents
    ]

    return entities

df['Named_Entities'] = df['Tokens'].apply(extract_ner)

print("Named Entity Recognition:")
print(df[['Tokens', 'POS_Tagging', 'Named_Entities']].head())
```

## RESULT

Thus, information extraction was performed successfully on the Amazon Fine Food Reviews dataset using spaCy. The reviews were loaded and preprocessed by converting the text to lowercase and removing punctuation. Tokenization, stopword removal, and filtering of non-alphabetic tokens were performed. Parts of Speech tagging and Named Entity Recognition were also carried out using spaCy.

# Ex6a: EXPLORATORY DATA ANALYSIS (EDA)

**Learning Objective:**

The objective of this experiment is to enable students to understand and perform Exploratory Data Analysis (EDA) on a real-world Netflix Titles dataset using Python.

**Problem Statement:**

Netflix maintains a large collection of movies and TV shows containing information such as title, content type, release year, date added, country, rating, duration, genre, director, cast, and description. The objective is to analyse the Netflix content library and identify important patterns using statistical techniques and visualisations.

## 1. Download Netflix Titles Dataset from Kaggle and Convert It to a DataFrame.

Python

```
#Mohanasankari A
#241801168

!pip install pandas numpy matplotlib seaborn wordcloud -q

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from wordcloud import WordCloud
import re
from google.colab import files

uploaded = files.upload()

df = pd.read_csv('netflix_titles.csv')
```

```
print("Dataset loaded successfully")
print(df.head())
```

## 2. Explore the Dataset.

Python

```
#Mohanasankari A
#241801168

print("First five rows:")
print(df.head())

print("\nLast five rows:")
print(df.tail())

print("\nDataset shape:")
print(df.shape)

print("\nColumn names:")
print(df.columns)

print("\nDataset information:")
df.info()

print("\nStatistical summary:")
print(df.describe(include='all'))

print("\nMissing values:")
print(df.isnull().sum())

print("\nDuplicate rows:")
print(df.duplicated().sum())
```

## 3. Content Distribution Analysis.

### a. How many Movies vs TV Shows are in the dataset?

```python
#Mohanasankari A
#241801168

content_counts = df['type'].value_counts()

print(content_counts)

plt.figure(figsize=(7, 5))
sns.barplot(x=content_counts.index, y=content_counts.values)
plt.title('Movies vs TV Shows on Netflix')
plt.xlabel('Content Type')
plt.ylabel('Number of Titles')
plt.show()

plt.figure(figsize=(6, 6))
plt.pie(content_counts.values, labels=content_counts.index,
        autopct='%1.1f%%', startangle=90)
plt.title('Content Type Distribution')
plt.show()
```

### b. What is the distribution of ratings (e.g., TV-MA, PG, R, etc.)?

```python
#Mohanasankari A
#241801168

rating_counts = df['rating'].value_counts()

print(rating_counts)

plt.figure(figsize=(12, 6))
sns.barplot(x=rating_counts.index, y=rating_counts.values)
plt.title('Distribution of Netflix Ratings')
plt.xlabel('Rating')
plt.ylabel('Number of Titles')
```

```
plt.xticks(rotation=45)
plt.show()
```

### c. Which countries have the most Netflix content?

Python

```python
#Mohanasankari A
#241801168

countries = df['country'].dropna().str.split(', ').explode()
top_countries = countries.value_counts().head(10)

print(top_countries)

plt.figure(figsize=(12, 6))
sns.barplot(x=top_countries.values, y=top_countries.index)
plt.title('Top 10 Countries Producing Netflix Content')
plt.xlabel('Number of Titles')
plt.ylabel('Country')
plt.show()
```

## 4. Time-Based Trend Analysis.

### a. In which years were the most shows/movies released?

Python

```python
#Mohanasankari A
#241801168

release_counts = df['release_year'].value_counts().sort_index()

print("Top release years:")
print(df['release_year'].value_counts().head(10))

plt.figure(figsize=(14, 6))
release_counts.plot(kind='line', marker='o')
```

```
plt.title('Netflix Titles by Release Year')
plt.xlabel('Release Year')
plt.ylabel('Number of Titles')
plt.show()
```

### b. In which years were titles added to Netflix?

Python

```
#Mohanasankari A
#241801168

df['date_added'] = pd.to_datetime(
    df['date_added'].str.strip(), errors='coerce'
)

df['year_added'] = df['date_added'].dt.year

added_counts = df['year_added'].value_counts().sort_index()

print("Titles added by year:")
print(added_counts)

plt.figure(figsize=(12, 6))
sns.barplot(x=added_counts.index, y=added_counts.values)
plt.title('Netflix Titles Added by Year')
plt.xlabel('Year Added')
plt.ylabel('Number of Titles')
plt.xticks(rotation=45)
plt.show()
```

### c. How has Netflix's content growth trend changed over time?

Python

```
#Mohanasankari A
#241801168
```

```python
growth = df.groupby('year_added').size().sort_index()

print(growth)

plt.figure(figsize=(12, 6))
plt.plot(growth.index, growth.values, marker='o')
plt.title('Netflix Content Growth Trend')
plt.xlabel('Year Added')
plt.ylabel('Number of Titles Added')
plt.grid(True)
plt.show()
```

## 5. Duration and Content Type Analysis.

### a. What is the distribution of movie durations (in minutes)?

Python

```python
#Mohanasankari A
#241801168

movies = df[df['type'] == 'Movie'].copy()

movies['duration_minutes'] = pd.to_numeric(
    movies['duration'].str.extract(r'(\d+)')[0],
    errors='coerce'
)

print(movies['duration_minutes'].describe())

plt.figure(figsize=(10, 6))
sns.histplot(movies['duration_minutes'].dropna(),
             bins=30, kde=True)
plt.title('Distribution of Movie Durations')
plt.xlabel('Duration (Minutes)')
plt.ylabel('Number of Movies')
plt.show()
```

### b. How many TV shows exist per number of seasons?

Python

```python
#Mohanasankari A
#241801168

tv_shows = df[df['type'] == 'TV Show'].copy()

tv_shows['seasons'] = pd.to_numeric(
    tv_shows['duration'].str.extract(r'(\d+)')[0],
    errors='coerce'
)

season_counts = tv_shows['seasons'].value_counts().sort_index()

print(season_counts)

plt.figure(figsize=(10, 6))
sns.barplot(x=season_counts.index, y=season_counts.values)
plt.title('Number of TV Shows by Seasons')
plt.xlabel('Number of Seasons')
plt.ylabel('Number of TV Shows')
plt.show()
```

## 6. Genre and Categories Analysis.

### a. What are the top 10 most common genres on Netflix?

Python

```python
#Mohanasankari A
#241801168

genres = df['listed_in'].dropna().str.split(', ').explode()
top_genres = genres.value_counts().head(10)

print(top_genres)

plt.figure(figsize=(12, 6))
```

```
sns.barplot(x=top_genres.values, y=top_genres.index)
plt.title('Top 10 Netflix Genres')
plt.xlabel('Number of Titles')
plt.ylabel('Genre')
plt.show()
```

### b. Which directors have the most titles on Netflix?

Python

```python
#Mohanasankari A
#241801168

directors = df['director'].dropna().str.split(', ').explode()
top_directors = directors.value_counts().head(10)

print(top_directors)

plt.figure(figsize=(12, 6))
sns.barplot(x=top_directors.values, y=top_directors.index)
plt.title('Top 10 Directors on Netflix')
plt.xlabel('Number of Titles')
plt.ylabel('Director')
plt.show()
```

### c. Which actors appear most frequently across Netflix shows/movies?

Python

```python
#Mohanasankari A
#241801168

actors = df['cast'].dropna().str.split(', ').explode()
top_actors = actors.value_counts().head(10)

print(top_actors)

plt.figure(figsize=(12, 6))
```

```
sns.barplot(x=top_actors.values, y=top_actors.index)
plt.title('Top 10 Actors on Netflix')
plt.xlabel('Number of Appearances')
plt.ylabel('Actor')
plt.show()
```

## 7. Text and Descriptions.

### a. What are the most common words in the description field? (Word Cloud)

Python

```
#Mohanasankari A
#241801168

text = ' '.join(df['description'].dropna().astype(str))

text = re.sub(r'[^a-zA-Z\s]', '', text).lower()

wordcloud = WordCloud(
    width=1000,
    height=500,
    background_color='white',
    stopwords=WordCloud().stopwords,
    max_words=100
).generate(text)

plt.figure(figsize=(14, 7))
plt.imshow(wordcloud, interpolation='bilinear')
plt.axis('off')
plt.title('Word Cloud of Netflix Descriptions')
plt.show()

from collections import Counter

words = text.split()
common_words = Counter(words).most_common(20)
```

```
print("Most common words:")
print(common_words)
```

## 8. In Which Months of the Year Does Netflix Add the Most Content?

Python

```
#Mohanasankari A
#241801168

df['month_added'] = df['date_added'].dt.month

month_counts = df['month_added'].value_counts().sort_index()

month_names = [
    'Jan', 'Feb', 'Mar', 'Apr', 'May', 'Jun',
    'Jul', 'Aug', 'Sep', 'Oct', 'Nov', 'Dec'
]

month_counts.index = [
    month_names[m - 1] for m in month_counts.index
]

print(month_counts.sort_values(ascending=False))

plt.figure(figsize=(12, 6))
sns.barplot(x=month_counts.index, y=month_counts.values)
plt.title('Netflix Content Added by Month')
plt.xlabel('Month')
plt.ylabel('Number of Titles')
plt.show()
```

## 9. Which Countries Produce More Movies vs TV Shows?

Python

```python
#Mohanasankari A
#241801168

country_data = df.dropna(subset=['country']).copy()

country_data['country'] = country_data['country'].str.split(', ')
country_data = country_data.explode('country')

country_type = pd.crosstab(
    country_data['country'],
    country_data['type']
)

top_country_names =
country_data['country'].value_counts().head(10).index

country_type_top = country_type.loc[
    country_type.index.intersection(top_country_names)
]

print(country_type_top)

country_type_top.plot(kind='bar', figsize=(12, 6))
plt.title('Movies vs TV Shows by Country')
plt.xlabel('Country')
plt.ylabel('Number of Titles')
plt.xticks(rotation=45)
plt.legend(title='Content Type')
plt.tight_layout()
plt.show()
```

## 10. What Are the Most Common Ratings for Movies vs TV Shows?

Python

```python
#Mohanasankari A
#241801168

rating_type = pd.crosstab(df['rating'], df['type'])

print(rating_type)

rating_type.plot(kind='bar', figsize=(14, 6))
plt.title('Ratings for Movies vs TV Shows')
plt.xlabel('Rating')
plt.ylabel('Number of Titles')
plt.xticks(rotation=45)
plt.legend(title='Content Type')
plt.tight_layout()
plt.show()
```

## 11. What Are the Oldest and Newest Titles Available on Netflix?

Python

```python
#Mohanasankari A
#241801168

oldest_title = df.loc[df['release_year'].idxmin()]
newest_title = df.loc[df['release_year'].idxmax()]

print("Oldest title:")
print(oldest_title[['title', 'type', 'release_year']])

print("\nNewest title:")
print(newest_title[['title', 'type', 'release_year']])

print("\nOldest release year:", df['release_year'].min())
print("Newest release year:", df['release_year'].max())
```

## 12. Which Genres Frequently Co-Occur Together? (Genre Correlation Heatmap)

Python

```python
#Mohanasankari A
#241801168

from itertools import combinations
from collections import Counter

genre_lists = (
    df['listed_in']
    .dropna()
    .apply(lambda x: [g.strip() for g in x.split(',')])
)

genre_pairs = Counter()

for genre_list in genre_lists:
    unique_genres = sorted(set(genre_list))

    for pair in combinations(unique_genres, 2):
        genre_pairs[pair] += 1

top_genres = [
    genre for genre, count in
    Counter(g for pair, count in genre_pairs.items()
            for g in pair).most_common(10)
]

cooccurrence = pd.DataFrame(
    0, index=top_genres, columns=top_genres
)

for (g1, g2), count in genre_pairs.items():
    if g1 in top_genres and g2 in top_genres:
        cooccurrence.loc[g1, g2] = count
```

```
        cooccurrence.loc[g2, g1] = count

plt.figure(figsize=(12, 8))
sns.heatmap(cooccurrence, annot=True, fmt='g', cmap='YlGnBu')
plt.title('Genre Co-occurrence Heatmap')
plt.xlabel('Genre')
plt.ylabel('Genre')
plt.show()

print("Genre co-occurrence matrix:")
print(cooccurrence)
```

## RESULT

Thus, Exploratory Data Analysis was performed successfully on the Netflix Titles dataset using Python. The dataset was explored and analysed to identify content distribution, ratings, country-wise content, release and addition trends, movie durations, TV show seasons, popular genres, directors, actors, description keywords, monthly additions, and genre co-occurrence patterns using suitable statistical methods and visualisations.

# EX6(b): TIME SERIES ANALYSIS AND FORECASTING USING ARIMA

## Learning Objective

The objective of this experiment is to understand and implement the fundamental concepts of time series data analysis and forecasting using Python.

## Problem Statement

An airline company maintains monthly records of the number of passengers travelling by air. The company wants to understand historical passenger trends and seasonal patterns and use this information to forecast future passenger demand.

As a Data Scientist, analyse the historical monthly airline passenger data, check stationarity, apply suitable transformations, build an ARIMA forecasting model, predict passenger numbers, and evaluate the accuracy of the forecasts.

## 1. Download Airline Passengers Dataset from GitHub.

Python

```
#Mohanasankari A

#241801168


!pip install pandas numpy matplotlib statsmodels scikit-learn -q


import pandas as pd

import numpy as np

import matplotlib.pyplot as plt


url =
"https://raw.githubusercontent.com/jbrownlee/Datasets/master/airlin
e-passengers.csv"


df = pd.read_csv(url)


print("Dataset downloaded successfully")

print(df.head())
```

## 2. Read the CSV with parse_dates and index_col so the time index becomes a DatetimeIndex.

Python

```python
#Mohanasankari A

#241801168


df = pd.read_csv(

    url,

    parse_dates=['Month'],

    index_col='Month'

)


df.index = pd.DatetimeIndex(df.index)


print(df.head())

print("\nIndex type:", type(df.index))

print("\nDataset information:")

print(df.info())
```

### 3. Explore the Dataset.

Python

```python
#Mohanasankari A

#241801168


print("First five rows:")
```

```
print(df.head())


print("\nLast five rows:")
print(df.tail())


print("\nDataset shape:")
print(df.shape)


print("\nColumn names:")
print(df.columns)


print("\nStatistical summary:")
print(df.describe())


print("\nMissing values:")
print(df.isnull().sum())
```

### 4. Plot the Series to Inspect Trend, Seasonality, and Outliers.

Python

```
#Mohanasankari A

#241801168
```

```
plt.figure(figsize=(12, 5))


plt.plot(df['Passengers'], marker='o')


plt.title('Monthly Airline Passenger Data')
plt.xlabel('Year')
plt.ylabel('Number of Passengers')
plt.grid(True)


plt.show()


from statsmodels.tsa.seasonal import seasonal_decompose


decomposition = seasonal_decompose(
    df['Passengers'],
    model='multiplicative',
    period=12
)


decomposition.plot()
plt.show()
```

### 5. Do Stationarity Check by Running Augmented Dickey-Fuller (ADF) Test.

Python

```python
#Mohanasankari A

#241801168


from statsmodels.tsa.stattools import adfuller


result = adfuller(df['Passengers'])


print("ADF Statistic:", result[0])

print("p-value:", result[1])


if result[1] <= 0.05:

    print("The series is stationary")

else:

    print("The series is non-stationary")
```

### 6. If the Series Is Non-Stationary, Convert It to Stationary by Differencing the Series.

Python

```python
#Mohanasankari A

#241801168
```

```
df['First_Difference'] = df['Passengers'].diff()


differenced_series = df['First_Difference'].dropna()


plt.figure(figsize=(12, 5))
plt.plot(differenced_series)


plt.title('First-Order Differenced Series')
plt.xlabel('Year')
plt.ylabel('Differenced Passenger Values')
plt.grid(True)


plt.show()
```

**7. Re-run ADF on the Differenced Series and Visualize the New DataFrame.**

Python

```
#Mohanasankari A
#241801168


result_diff = adfuller(differenced_series)


print("ADF Statistic:", result_diff[0])
```

```
print("p-value:", result_diff[1])


if result_diff[1] <= 0.05:
    print("The differenced series is stationary")
else:
    print("The differenced series is non-stationary")


print("\nDifferenced DataFrame:")
print(df[['Passengers', 'First_Difference']].head(15))
```

**8. If Still Non-Stationary, Consider Another Difference (d=2) or Log Transformation Before Differencing.**

Python

```
#Mohanasankari A
#241801168


d = 1


if result_diff[1] > 0.05:
    log_series = np.log(df['Passengers'])


    log_diff = log_series.diff().dropna()
```

```
    result_log = adfuller(log_diff)


    print("Log-differenced ADF Statistic:", result_log[0])

    print("Log-differenced p-value:", result_log[1])


    if result_log[1] <= 0.05:

        d = 1

        print("Log transformation followed by first differencing is
stationary")

    else:

        second_diff = differenced_series.diff().dropna()

        result_second = adfuller(second_diff)


        print("Second-difference ADF Statistic:", result_second[0])

        print("Second-difference p-value:", result_second[1])


        if result_second[1] <= 0.05:

            d = 2

            print("Second differencing achieved stationarity")

        else:

            d = 2

            print("Series may need further analysis; using d=2")
```

```
else:
    print("First differencing achieved stationarity")

print("Selected differencing order d =", d)
```

### 9. Split the Dataset for Training and Testing on the Original Series.

Python

```
#Mohanasankari A
#241801168

series = df['Passengers']

train_size = int(len(series) * 0.8)

train = series.iloc[:train_size]
test = series.iloc[train_size:]

print("Training data size:", len(train))
print("Testing data size:", len(test))

print("\nTraining data:")
```

```
print(train.tail())


print("\nTesting data:")
print(test.head())
```

### 10. Fit ARIMA Model on Training Set. The d Value Comes from Step 8.

Python

```
#Mohanasankari A
#241801168


from statsmodels.tsa.arima.model import ARIMA


model = ARIMA(
    train,
    order=(2, d, 2)
)


model_fit = model.fit()


print(model_fit.summary())
```

### 11. Perform Forecasts on the Test Dataset.

Python

```python
#Mohanasankari A

#241801168


forecast = model_fit.forecast(steps=len(test))


forecast = pd.Series(

    forecast,

    index=test.index,

    name='Predicted_Passengers'

)


print("Forecasted Passenger Numbers:")

print(forecast)


comparison = pd.DataFrame({

    'Actual': test,

    'Predicted': forecast

})


print("\nActual vs Predicted:")

print(comparison)
```

### 12. Compute Metrics MSE, MAE, and RMSE on the Predictions.

Python

```
#Mohanasankari A

#241801168


from sklearn.metrics import mean_squared_error, mean_absolute_error


mse = mean_squared_error(test, forecast)

mae = mean_absolute_error(test, forecast)

rmse = np.sqrt(mse)


print("Mean Squared Error (MSE):", mse)

print("Mean Absolute Error (MAE):", mae)

print("Root Mean Squared Error (RMSE):", rmse)
```

### 13. Plot Train, Test, and Predictions in a Single Plot.

Python

```
#Mohanasankari A

#241801168


plt.figure(figsize=(12, 6))


plt.plot(train.index, train,
```

```
         label='Training Data')


plt.plot(test.index, test,
         label='Actual Test Data')


plt.plot(forecast.index, forecast,
         label='ARIMA Forecast',
         linestyle='--')


plt.title('Airline Passenger Forecast using ARIMA')
plt.xlabel('Year')
plt.ylabel('Number of Passengers')


plt.legend()
plt.grid(True)
plt.show()
```

## RESULT

Thus, time series analysis and forecasting were performed successfully on the Airline Passengers dataset using Python. The time series was visualised, stationarity was checked using the Augmented Dickey-Fuller test, and differencing was applied as required. An ARIMA model was fitted to the training data, passenger numbers were forecast for the test period, and the prediction accuracy was evaluated using MSE, MAE, and RMSE. The training data, actual test values, and predicted values were visualised in a single plot.

# Ex7: MODEL PLANNING AND BUILDING – CLUSTERING

## Learning Objective

The objective of this experiment is to understand and implement the K-Means clustering algorithm for customer segmentation.

## Problem Statement

A retail company wants to group its customers based on their Annual Income and Spending Score to design personalized marketing campaigns. The K-Means clustering algorithm is used to identify customer segments, visualize the clusters, and recommend suitable marketing strategies.

### 1. Use Mall Customer Segmentation Dataset from Kaggle and Load It into a Pandas DataFrame.

None

```
#Mohanasankari A
#241801168

!pip install pandas numpy matplotlib seaborn scikit-learn -q

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from google.colab import files
uploaded = files.upload()

df = pd.read_csv('Mall_Customers.csv')

print("Dataset loaded successfully")
print(df.head())
```

## 2. Explore the Dataset.

Display the first and last five records, check the dataset shape and data types, identify missing values and duplicate records, and display summary statistics.

None

```
#Mohanasankari A
#241801168

print("First five records:")
print(df.head())

print("\nLast five records:")
print(df.tail())

print("\nDataset shape:")
print(df.shape)

print("\nData types:")
print(df.dtypes)

print("\nMissing values:")
print(df.isnull().sum())

print("\nDuplicate records:")
print(df.duplicated().sum())

print("\nSummary statistics:")
print(df.describe(include='all'))
```

## 3. Select the Important Features for Clustering: Annual Income (k$), Spending Score (1–100).

None

```
#Mohanasankari A
#241801168

X = df[['Annual Income (k$)', 'Spending Score (1-100)']]
```

```
print("Selected features:")
print(X.head())

print("\nFeature information:")
print(X.info())
```

### 4. Standardize the Selected Features Using StandardScaler for Clustering.

```
None
#Mohanasankari A
#241801168

from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()

X_scaled = scaler.fit_transform(X)

X_scaled_df = pd.DataFrame(
    X_scaled,
    columns=['Annual Income (Scaled)', 'Spending Score (Scaled)']
)

print("Standardized features:")
print(X_scaled_df.head())
```

### 5. Visualize the Selected Features Using Scatter Plot and Pair Plot.

#### a. Scatter Plot

```
None
#Mohanasankari A
#241801168

plt.figure(figsize=(8, 6))
```

```
plt.scatter(
    X['Annual Income (k$)'],
    X['Spending Score (1-100)']
)

plt.title('Annual Income vs Spending Score')
plt.xlabel('Annual Income (k$)')
plt.ylabel('Spending Score (1-100)')
plt.grid(True)

plt.show()
```

**b. Pair Plot**

```
None
#Mohanasankari A
#241801168

sns.pairplot(
    df[['Annual Income (k$)', 'Spending Score (1-100)']]
)

plt.suptitle('Pair Plot of Customer Features', y=1.02)
plt.show()
```

# MODEL PLANNING AND BUILDING

### 6. Determine the Optimal Number of Clusters Using the Elbow Method. Plot the Elbow Curve and Identify the Optimal Value of K.

```
None
#Mohanasankari A
#241801168
```

```
from sklearn.cluster import KMeans

wcss = []

for k in range(1, 11):
    kmeans = KMeans(
        n_clusters=k,
        init='k-means++',
        random_state=42,
        n_init=10
    )

    kmeans.fit(X_scaled)
    wcss.append(kmeans.inertia_)

plt.figure(figsize=(8, 5))

plt.plot(range(1, 11), wcss, marker='o')

plt.title('Elbow Method for Optimal K')
plt.xlabel('Number of Clusters (K)')
plt.ylabel('WCSS')
plt.xticks(range(1, 11))
plt.grid(True)

plt.show()
```

**Observation:** The optimal number of clusters is selected by identifying the elbow point, where the decrease in WCSS starts slowing down. For this dataset, (K=5) is a commonly used choice.

### 7. Train the K-Means Clustering Model Using the Optimal Number of Clusters. Assign Cluster Labels to Each Customer, Add Them as a New Column, and Save the Dataset as a File.

```
None
#Mohanasankari A
#241801168
```

```
optimal_k = 5

kmeans = KMeans(
    n_clusters=optimal_k,
    init='k-means++',
    random_state=42,
    n_init=10
)

df['Cluster'] = kmeans.fit_predict(X_scaled)

print("Cluster labels assigned successfully")
print(df.head())

df.to_csv('Mall_Customers_Clustered.csv', index=False)

print("\nClustered dataset saved successfully")

from google.colab import files
files.download('Mall_Customers_Clustered.csv')
```

## CLUSTER VISUALIZATION

**8. Visualize the Customer Clusters Using a Scatter Plot. Use Annual Income on the X-axis, Spending Score on the Y-axis, Different Colors for Each Cluster, Highlight Cluster Centroids, and Display Their Coordinates.**

None

```
#Mohanasankari A
#241801168

centroids_scaled = kmeans.cluster_centers_

centroids_original = scaler.inverse_transform(centroids_scaled)
```

```
centroids_df = pd.DataFrame(
    centroids_original,
    columns=['Annual Income (k$)', 'Spending Score (1-100)']
)

plt.figure(figsize=(10, 7))

sns.scatterplot(
    data=df,
    x='Annual Income (k$)',
    y='Spending Score (1-100)',
    hue='Cluster',
    palette='tab10',
    s=80
)

plt.scatter(
    centroids_df['Annual Income (k$)'],
    centroids_df['Spending Score (1-100)'],
    marker='X',
    s=250,
    color='black',
    label='Centroids'
)

for i, row in centroids_df.iterrows():
    plt.annotate(
        f"C{i}: ({row.iloc[0]:.1f}, {row.iloc[1]:.1f})",
        (row.iloc[0], row.iloc[1]),
        xytext=(5, 5),
        textcoords='offset points'
    )

plt.title('Customer Segmentation Using K-Means')
plt.xlabel('Annual Income (k$)')
plt.ylabel('Spending Score (1-100)')
```

```
plt.legend()
plt.grid(True)
plt.show()

print("Cluster centroid coordinates:")
print(centroids_df.round(2))
```

# CLUSTER ANALYSIS

### 9. Calculate the Number of Customers in Each Cluster.

```
None
#Mohanasankari A
#241801168

cluster_counts = df['Cluster'].value_counts().sort_index()

print("Number of customers in each cluster:")
print(cluster_counts)

plt.figure(figsize=(8, 5))

sns.barplot(
    x=cluster_counts.index,
    y=cluster_counts.values
)

plt.title('Number of Customers in Each Cluster')
plt.xlabel('Cluster')
plt.ylabel('Number of Customers')
plt.show()
```

### 10. Compute the Average Annual Income and Spending Score for Each Cluster.

None

```
#Mohanasankari A
#241801168

cluster_analysis = df.groupby('Cluster').agg(
    Customer_Count=('CustomerID', 'count'),
    Average_Annual_Income=('Annual Income (k$)', 'mean'),
    Average_Spending_Score=('Spending Score (1-100)', 'mean')
).round(2)

print("Cluster Analysis:")
print(cluster_analysis)
```

## MARKETING STRATEGIES

None

```
#Mohanasankari A
#241801168

for cluster, row in cluster_analysis.iterrows():
    income = row['Average_Annual_Income']
    spending = row['Average_Spending_Score']

    print(f"\nCluster {cluster}:")
    print(f"Average Annual Income: {income}")
    print(f"Average Spending Score: {spending}")

    if income >= 60 and spending >= 60:
        print("Strategy: Offer loyalty rewards and premium
products.")
    elif income >= 60 and spending < 40:
        print("Strategy: Provide targeted discounts and exclusive
offers.")
    elif income < 40 and spending >= 60:
        print("Strategy: Offer affordable premium deals and reward
points.")
    elif income < 40 and spending < 40:
```

```
        print("Strategy: Use budget-friendly promotions to increase
engagement.")
    else:
        print("Strategy: Recommend personalized offers and
cross-selling.")
```

## RESULT

Thus, customer segmentation was performed successfully using the K-Means clustering algorithm. The Mall Customer Segmentation dataset was explored, the Annual Income and Spending Score features were standardized, and the optimal number of clusters was investigated using the Elbow Method. Customers were grouped into clusters, the cluster centroids were visualized, and the number of customers and average feature values were calculated for each cluster. Suitable marketing strategies were recommended based on the identified customer segments.

# EX8: INTERACTIVE DASHBOARDS WITH TABLEAU

## Learning Objective

The objective of this experiment is to understand and implement interactive data visualisation and dashboard design using Tableau.

## Problem Statement

Create an interactive dashboard in Tableau using the Sample – Superstore dataset from Kaggle. The dashboard should include a bar chart showing Sales by Category, a line chart showing Sales trends over time, a scatter plot showing Profit vs Sales by Sub-Category, and interactive filters and dashboard actions.

## 1. Open Tableau and Load Dataset

**Procedure:**

1. Open Tableau Desktop or Tableau Public.
2. Download the Sample Superstore dataset from Kaggle.
3. Select **Connect** → **To a File** → **Microsoft Excel** or **Text File**, depending on the downloaded file format.
4. Select the downloaded dataset file.
5. Check the data preview and click **Sheet 1** to begin creating visualisations.

**Dataset:** Sample Superstore

**Important fields:** Order Date, Category, Sub-Category, Region, Sales, Profit, Segment, and Quantity.

## 2. Explore the Data

**Procedure:**

1. Check the field names and data types in the Data pane.
2. Ensure that Order Date is recognised as a Date field.
3. Ensure that Sales, Profit, and Quantity are recognised as numerical measures.

4. Check for missing values and verify that Category, Sub-Category, and Region are available.
5. Drag Sales and Profit into a worksheet to explore their aggregated values.

**Expected Output:**

The dataset is loaded successfully, and the required dimensions and measures are available for creating charts.

# 3. Create Worksheet 1 – Bar Chart (Sales by Category)

**Procedure:**

1. Click the Sheet 1 tab and rename it **Bar_Sales_by_Category**.
2. Drag **Category** to Columns.
3. Drag **Sales** to Rows. Ensure the aggregation is SUM(Sales).
4. Sort the bars in descending order using the sort icon.
5. Drag **Region** to Color in the Marks card.
6. Click Label and select **Show Mark Labels**.
7. Optionally, drag Region to Filters, select all regions, and choose **Show Filter**.

**Expected Output:**

A bar chart displaying total sales for Furniture, Office Supplies, and Technology. Different colours represent the regions, and the bar labels display the sales values.

# 4. Create Worksheet 2 – Line Chart (Sales Trend Over Time)

**Procedure:**

1. Click New Worksheet and rename it **Line_Sales_Trend**.
2. Drag **Order Date** to Columns.
3. Open the Order Date pill menu and select **Month (continuous)**, displayed as a green pill.
4. Drag **Sales** to Rows, ensuring SUM(Sales).
5. Drag **Region** to Color to create separate lines for each region.
6. Right-click the vertical axis, select Format, and format the values as currency if required.
7. Optionally, add a Region or Category filter and select Show Filter.

**Expected Output:**

A line chart showing monthly sales trends over time, with separate coloured lines for each region. The chart helps identify sales growth, declines, and seasonal patterns.

# 5. Create Worksheet 3 – Scatter Plot (Profit vs Sales)

**Procedure:**

1. Click New Worksheet and rename it **Scatter_Profit_vs_Sales**.
2. Drag **Sales** to Columns and ensure SUM(Sales).
3. Drag **Profit** to Rows and ensure SUM(Profit).
4. Drag **Sub-Category** to Detail on the Marks card so that each point represents a sub-category.
5. Drag **Category** to Color.
6. Optionally, drag Segment to Shape.
7. Open the Analytics pane and drag an **Average Line** onto the Table for the horizontal and vertical axes, where supported.

**Expected Output:**

A scatter plot displaying the relationship between Sales and Profit for different sub-categories. Points above the zero-profit level represent profitable sub-categories, while points below it represent loss-making sub-categories.

# 6. Build the Dashboard

**Procedure:**

1. Click **New Dashboard** at the bottom of the Tableau window.
2. Rename the dashboard **Sales Dashboard**.
3. Set the dashboard size to approximately **1200 × 800 pixels**.
4. From the Sheets pane, drag the three worksheets onto the dashboard.
5. Place the Line_Sales_Trend chart at the top.
6. Place Bar_Sales_by_Category at the bottom-left.
7. Place Scatter_Profit_vs_Sales at the bottom-right.
8. Use horizontal and vertical layout containers to align the charts.
9. Add a dashboard title: **Sales and Profit Dashboard**.
10. Adjust chart sizes, legends, labels, and spacing for readability.

**Expected Output:**

A single dashboard containing the sales trend line chart, category sales bar chart, and profit-versus-sales scatter plot in a well-organised layout.

# 7. Add Interactivity

## a. Add Filters

**Procedure:**

1. Open the dashboard.
2. Click the dropdown arrow at the top-right of a worksheet.
3. Select **Filters** → **Region** or choose Show Filter for an available field.
4. Repeat for Category or Order Date, as required.
5. Configure the filter card to apply to the relevant worksheets using **Apply to Worksheets** → **Selected Worksheets** or **All Using This Data Source**.

**Expected Output:**

Users can select a region, category, or date range to filter the dashboard and analyse specific parts of the dataset.

## b. Add Dashboard Filter Action

**Procedure:**

1. Select **Dashboard** → **Actions** → **Add Action** → **Filter**.
2. Set the source sheet to **Bar_Sales_by_Category**.
3. Select **Line_Sales_Trend** and **Scatter_Profit_vs_Sales** as target sheets.
4. Set Run action on to **Select**.
5. Set Clearing the selection to **Show all values**.
6. Click OK to save the action.

**Expected Output:**

Selecting a category or bar in the bar chart filters the target charts according to the selected data. Clear the selection to restore all values.

## c. Add Highlight Action (Optional)

**Procedure:**

1. Select **Dashboard** → **Actions** → **Add Action** → **Highlight**.
2. Select the source and target worksheets.
3. Choose Sub-Category or Category as the highlighting field.
4. Configure the action to run on Hover.
5. Click OK.

**Expected Output:**

Hovering over a relevant data mark highlights matching data in other charts, making it easier to compare related information.

# 8. Publish or Export

**Procedure:**

**Option A – Publish to Tableau Public**

1. Save the workbook.
2. Select the Tableau Public publishing option available in your version of Tableau.
3. Sign in to your Tableau Public account.
4. Enter the workbook name and publish it.
5. Open the published dashboard and verify that the filters and actions work.

**Option B – Export as PDF or Image**

1. Open the completed dashboard.
2. Select **File → Export as PDF** or the appropriate image-export option available in your Tableau version.
3. Choose the dashboard and required layout settings.
4. Save the exported file.
5. Open the exported file and verify that the charts and dashboard title are visible.

**Expected Output:**

The completed dashboard is published online or exported as a PDF/image for documentation and presentation.

# OUTPUT

The following visualisations are obtained after completing the experiment:

**Output 1 – Bar Chart: Sales by Category**

A bar chart showing total sales for Furniture, Office Supplies, and Technology, with regional colour coding and sales labels.

**Output 2 – Line Chart: Sales Trend Over Time**

A line chart showing monthly sales trends, with separate lines for different regions.

**Output 3 – Scatter Plot: Profit vs Sales**

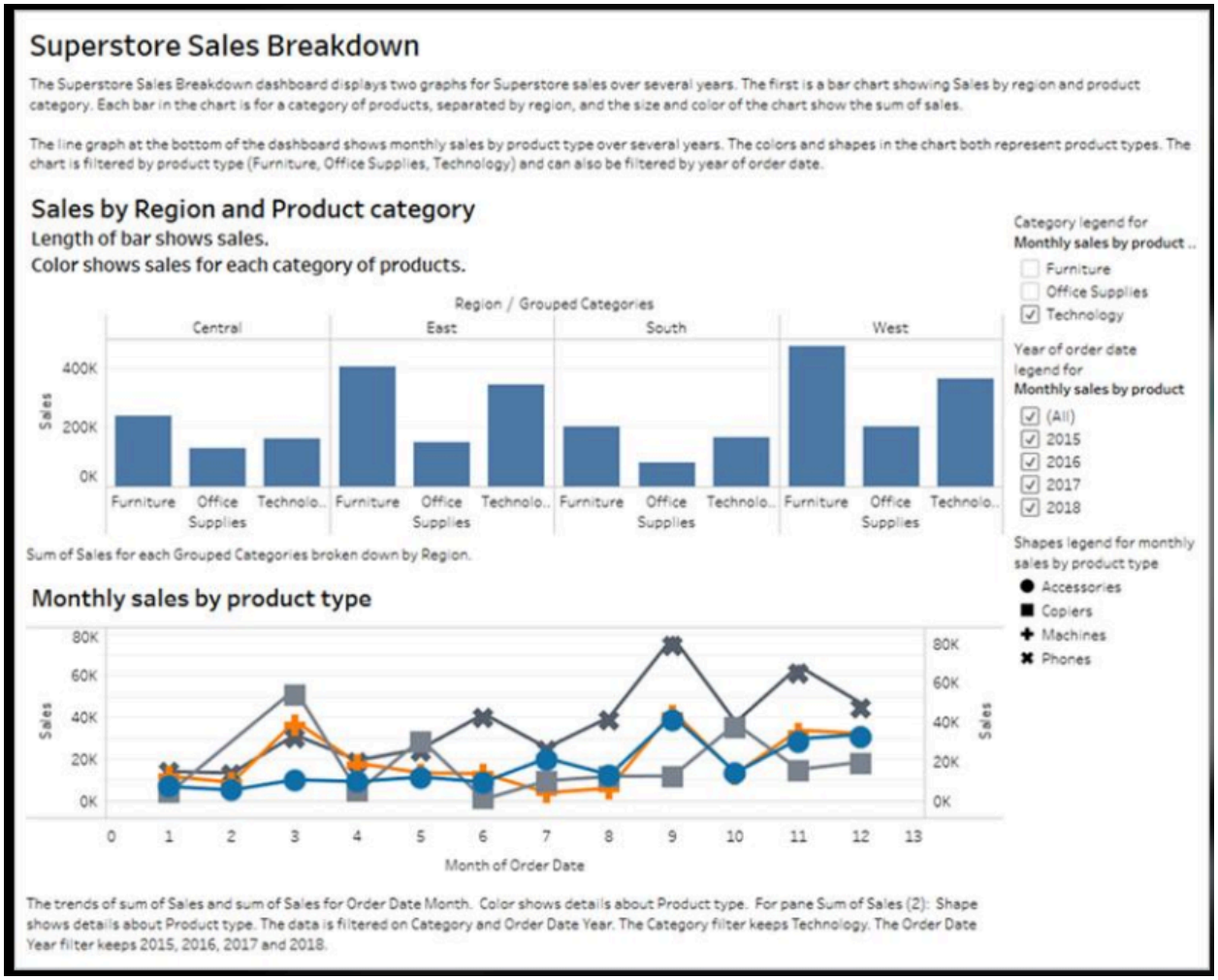
A scatter plot displaying sales and profit for each sub-category, helping identify profitable and loss-making categories.

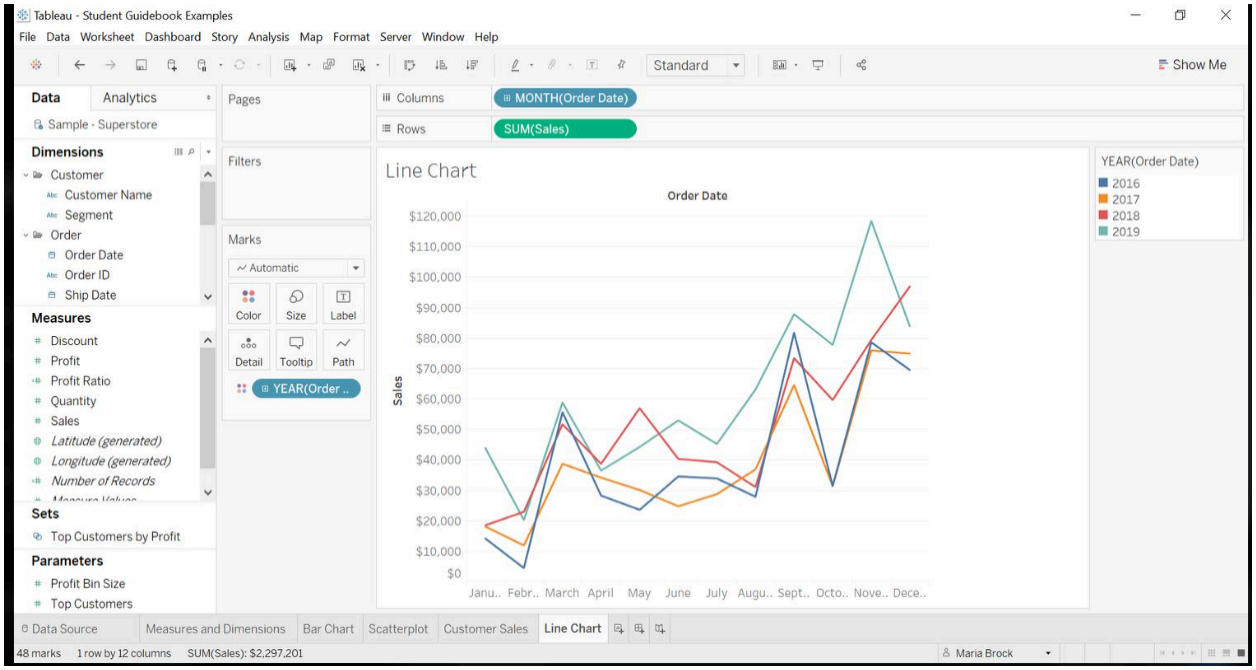
**Output 4 – Interactive Sales and Profit Dashboard**

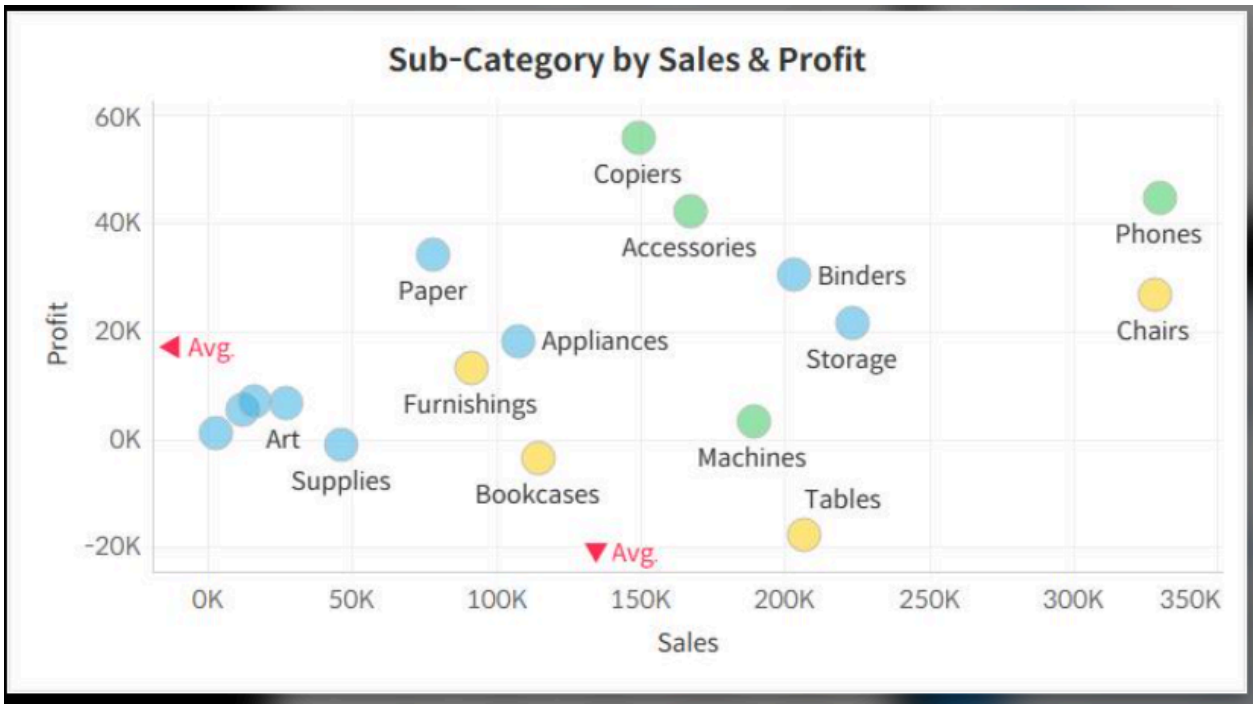
A dashboard containing all three charts, interactive filters, and dashboard actions. Selecting a category or region allows users to explore related sales and profit trends.

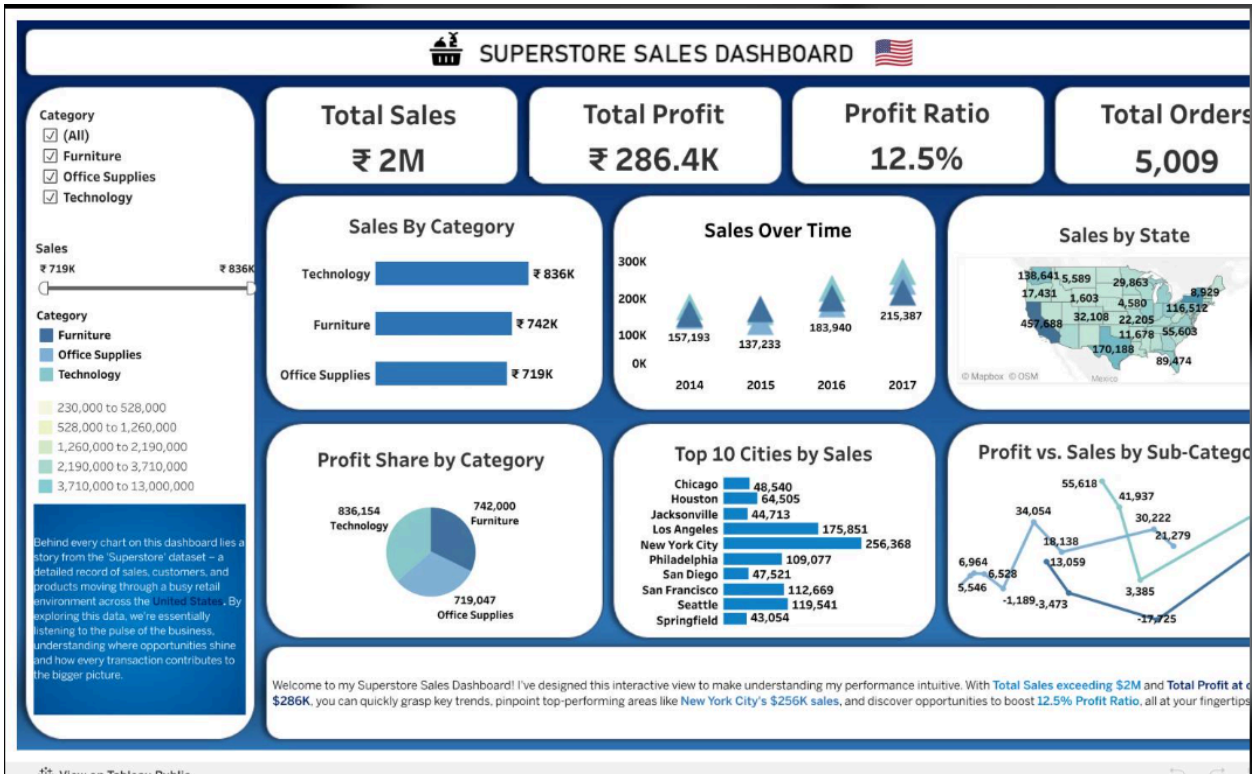
# RESULT

Thus, an interactive dashboard was created in Tableau using the Sample Superstore dataset. A bar chart, line chart, and scatter plot were developed to analyse category-wise sales, sales trends over time, and the relationship between sales and profit. Interactive filters and dashboard actions were configured to enable dynamic data exploration. The dashboard was then published or exported for reporting and presentation.




Tableau - Student Guidebook Examples
File Data Worksheet Dashboard Story Analysis Map Format Server Window Help
Standard
Show Me
Data
Analytics
Sample - Superstore
Dimensions
Customer
Customer Name
Segment
Order
Order Date
Order ID
Ship Date
Measures
Discount
Profit
Profit Ratio
Quantity
Sales
Latitude (generated)
Longitude (generated)
Number of Records
Sets
Top Customers by Profit
Parameters
Profit Bin Size
Top Customers
Pages
Filters
Marks
Automatic
Color
Size
Label
Detail
Tooltip
Path
YEAR(Order ..
Columns
MONTH(Order Date)
Rows
SUM(Sales)
Line Chart
Order Date
Sales
$120,000
$110,000
$100,000
$90,000
$80,000
$70,000
$60,000
$50,000
$40,000
$30,000
$20,000
$10,000
$0
Janu.. Febr.. March April May June July Augu.. Sept.. Octo.. Nove.. Dece..
YEAR(Order Date)
2016
2017
2018
2019
Data Source
Measures and Dimensions
Bar Chart
Scatterplot
Customer Sales
Line Chart
48 marks
1 row by 12 columns
SUM(Sales): $2,297,201
Maria Brock


Sub-Category by Sales & Profit
Profit
60K
40K
20K
0K
-20K
Avg.
Avg.
Copiers
Accessories
Paper
Binders
Phones
Chairs
Appliances
Storage
Furnishings
Art
Supplies
Machines
Bookcases
Tables
0K
50K
100K
150K
200K
250K
300K
350K
Sales


SUPERSTORE SALES DASHBOARD
Category
(All)
Furniture
Office Supplies
Technology
Sales
₹ 719K
₹ 836K
Category
Furniture
Office Supplies
Technology
230,000 to 528,000
528,000 to 1,260,000
1,260,000 to 2,190,000
2,190,000 to 3,710,000
3,710,000 to 13,000,000
Behind every chart on this dashboard lies a story from the 'Superstore' dataset – a detailed record of sales, customers, and products moving through a busy retail environment across the United States. By exploring this data, we're essentially listening to the pulse of the business, understanding where opportunities shine and how every transaction contributes to the bigger picture.
Total Sales
₹ 2M
Total Profit
₹ 286.4K
Profit Ratio
12.5%
Total Orders
5,009
Sales By Category
Technology
₹ 836K
Furniture
₹ 742K
Office Supplies
₹ 719K
Sales Over Time
300K
200K
100K
0K
157,193
137,233
183,940
215,387
2014
2015
2016
2017
Sales by State
138,641
5,589
29,863
8,929
17,431
1,603
4,580
116,512
457,688
32,108
22,205
11,678
55,603
170,188
89,474
© Mapbox © OSM
Mexico
Profit Share by Category
836,154
Technology
742,000
Furniture
719,047
Office Supplies
Top 10 Cities by Sales
Chicago 48,540
Houston 64,505
Jacksonville 44,713
Los Angeles 175,851
New York City 256,368
Philadelphia 109,077
San Diego 47,521
San Francisco 112,669
Seattle 119,541
Springfield 43,054
Profit vs. Sales by Sub-Catego
55,618
41,937
34,054
30,222
21,279
18,138
13,059
6,964
6,528
5,546
-1,189
-3,473
3,385
-17,725
Welcome to my Superstore Sales Dashboard! I've designed this interactive view to make understanding my performance intuitive. With Total Sales exceeding $2M and Total Profit at
$286K, you can quickly grasp key trends, pinpoint top-performing areas like New York City's $256K sales, and discover opportunities to boost 12.5% Profit Ratio, all at your fingertips